

Course: Object Oriented Programming

Lab#3

Constructors, Static Variable & Static Method

Task 1:

Create a Movie class with the following:

1. Fields:

- title (String): Title of the movie.
- genre (String): Genre of the movie.
- rating (double): Rating of the movie.
- isSuitableForChildren (boolean): A field that determines if the movie is suitable for children.

2. Constructor:

- Initializes all the fields.

3. Methods:

- checkSuitability(): A method that returns true if the rating is less than or equal to 8.0, otherwise false.

4. Main Method:

- Create at least **two movie objects** and display their details in the format:
"Movie: [title], Genre: [genre], Rating: [rating], Suitable for Children: [Yes/No]".

Sample Output:

```
Movie: "The Legend of Maula Jatt", Genre: Action, Rating: 9.2, Suitable for Children: No
Movie: "3 Bahadur", Genre: Animation, Rating: 7.8, Suitable for Children: Yes
```

Task 2:

Create a BankAccount class with the following:

1. Fields:

- accountNumber (int): Account number.
- accountHolder (String): Name of the account holder.
- balance (double): Account balance.

2. Static Field:

- accountCounter: Tracks the number of accounts created.

3. Constructor:

- Initializes accountNumber, accountHolder, and balance.
- Increments the accountCounter.

4. Methods:

- deposit(double amount): Adds the specified amount to the balance.
- withdraw(double amount): Reduces the balance by the specified amount, but ensures the balance does not fall below **1,000 PKR**. Display a message if the withdrawal is not possible.
- getTotalAccounts(): A static method that displays the total number of accounts created.

5. Main Method:

- Create at least **three BankAccount objects**.
- Perform deposit and withdrawal operations on each account.
- Display the account details after each transaction in the format:
"Account Number: [accountNumber], Account Holder: [accountHolder], Balance: [balance] PKR".

Sample Output:

```
Account Number: 101, Account Holder: Ahmed Khan, Balance: 20,000 PKR
After Withdrawal of 5,000 PKR:
Account Number: 101, Account Holder: Ahmed Khan, Balance: 15,000 PKR
Total Bank Accounts: 3
```

Task 3:

Create a Product class with the following:

1. Fields:

- productName (String): Name of the product.
- price (double): Original price of the product.

2. Static Field:

- discount (double): Global discount percentage applied to all products.

3. Constructor:

- Initializes productName and price.

4. Methods:

- setDiscount(double discount): A static method to set the global discount percentage.
- calculateDiscountedPrice(): Calculates and returns the discounted price of the product.
- compareDiscounts(Product other): Compares the discounted prices of two products and returns which one has a higher discounted price.

5. Main Method:

- Create at least **two Product objects**.
- Apply a global discount using the setDiscount() method.
- Display the original and discounted prices for each product in the format:
"Product: [productName], Original Price: [price], Discounted Price: [discountedPrice]".
- Use the compareDiscounts() method to display which product has the higher discounted price.

Sample Output:

```
Product: Rooh Afza, Original Price: 1,200 PKR, Discounted Price: 960 PKR
Product: Shan Masala, Original Price: 600 PKR, Discounted Price: 510 PKR
Product "Rooh Afza" has a higher discounted price!
```

Task 4:

Create a Student class with the following:

1. Fields:

- studentName (String): Name of the student.
- marks (int[]): Array of marks for various subjects.

2. Constructor:

- Accepts the student's name and marks array as parameters.

3. Static Method:

- calculateGPA(int[] marks): Calculates the GPA by averaging the marks and scaling them to a maximum of 4.0.
 1. Formula: $GPA = (\text{sum of marks} / \text{total subjects}) / 25$.
 2. GPA should be capped at 4.0.

4. Instance Methods:

- getGrades(): Returns an array of grades (A, B, C, D, F) based on the marks for each subject.
- displayDetails(): Displays the student's name, GPA, and grades in the format: "Student: [studentName], GPA: [gpa], Grades: [grade1, grade2, ...]".

5. Main Method:

- Prompt the user for the number of students.
- For each student:
 1. Ask for their name and the number of subjects.
 2. Input marks for all subjects.
 3. Create a Student object and store it in an array.
- Display the details of each student.
- Identify and display the top performer based on GPA.

Sample Output:

```
Student: Ali Raza, GPA: 3.52, Grades: [B, A, C]
Student: Hina Khan, GPA: 3.68, Grades: [A, B, A]
Top Performer: Hina Khan
```

Task 5:

Create an Order class with the following:

1. **Fields:**

- orderId (int): Unique identifier for the order.
- productName (String): Name of the product.
- quantity (int): Quantity of the product ordered.
- pricePerUnit (double): Price per unit of the product.

2. **Static Fields:**

- totalOrders (int): Counts the total number of orders.
- totalRevenue (double): Tracks the total revenue generated by all orders.

3. **Constructor:**

- Initializes the instance fields.
- Increments totalOrders.
- Updates totalRevenue based on the total price of the order.

4. **Methods:**

- getTotalOrders(): A static method to display the total number of orders placed.
- displayOrderDetails(): Displays the details of an order, including the total price.
- Apply a **bulk discount** of 10% if the quantity exceeds 50 items.

5. **Main Method:**

- Create at least **three Order objects**.
- Display the details of each order and the total orders and revenue at the end.

Sample Output:

```
Order ID: 1, Product: Chaye Patti, Quantity: 30, Total Price: 3,000 PKR
Order ID: 2, Product: Biryani Masala, Quantity: 60, Total Price after Bulk Discount: 5,400
Order ID: 3, Product: Olper's Milk, Quantity: 10, Total Price: 1,500 PKR
Total Orders: 3
Total Revenue: 9,900 PKR
```